DSU W23 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

WINTER – 2023 EXAMINATION
MODEL ANSWER

Subject: Data Structure Using C **Subject Code:** **22317**

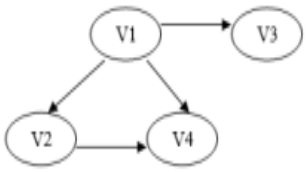
Important Instructions to examiners:

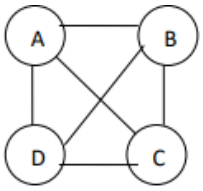
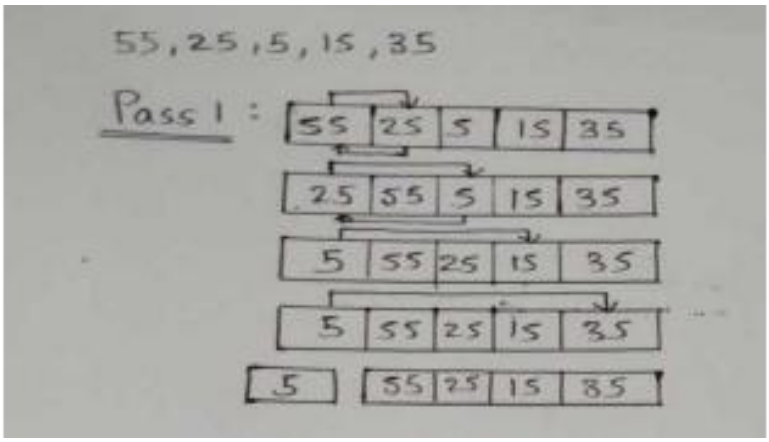
1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more Importance (Not applicable for subject English and Communication Skills.
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for anyequivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.
8) As per the policy decision of Maharashtra State Government, teaching in English/Marathi and Bilingual (English + Marathi) medium is introduced at first year of AICTE diploma Programme from academic year 2021-2022. Hence if the students in first year (first and second semesters) write answers in Marathi or bilingual language (English +Marathi), the Examiner shall consider the same and assess the answer based on matching of concepts with model answer.

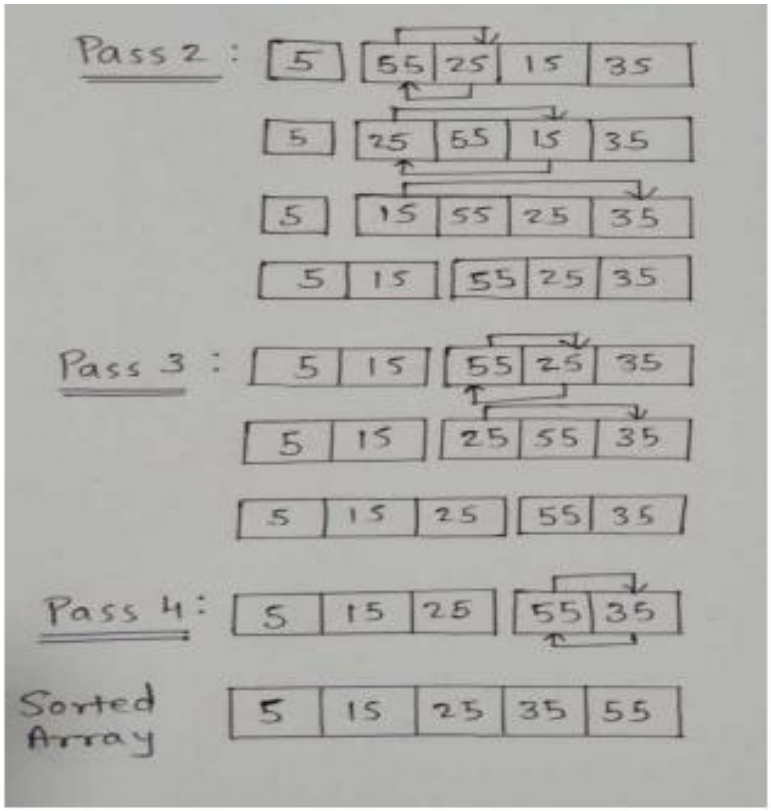
<table>
<tr><th>Q. No</th><th>Sub Q.N.</th><th>Answer</th><th>Marking Scheme</th></tr>
<tr><td>1.</td><td>a)

Ans.</td><td>Attempt any <u>FIVE</u> of the following:
State any two differences between linear search and binary search
<table>
<tr><th>Sr. No</th><th>Linear Search</th><th>Binary Search</th></tr>
<tr><td>1.</td><td>In this type, search element is compared with all array elements starting with 0th position element till search elements matches to array element or till the end of array list.</td><td>In this type, search element is compared with only mid element till search element matches array element or only one element remains in array list for comparison.</td></tr>
<tr><td>2.</td><td>It works on sorted as well as unsorted array list.</td><td>It works only on sorted array list.</td></tr>
</table></td><td>10
2M
Any two differences 1M each</td></tr>
</table>

		3. Number of comparisons is more than binary search for large dataset. | Number of comparisons is less than linear search for large dataset. 4. Speed of searching is slow as search element is compared with all elements. | Speed of searching is fast as search element is compared only with mid element.	
	b) Ans.	**Define term pointer and null pointer** **Pointer** Pointer is a variable which is used to store the address of another variable of same data type. **Null pointer** A Null Pointer is a pointer that does not point to any memory location. The null pointer stores the Null value.	**2M** ***Each definition 1M***
	c) Ans.	**Define the terms: Linear data structure and non-linear data structure** **Linear Data Structure** A data structure in which all data elements are stored in a particular sequence is known as linear data structure. **Non-Linear data structure** A data structure in which all data elements are not stored in any particular sequence is known as nonlinear data structure.	**2M** ***Definition of each term 1M***
	d) Ans.	**List any four applications of queue** **Applications of Queue:** • Playlist on device • Managing requests on a single shared resource such as CPU scheduling and disk scheduling. • Handling hardware or real-time systems interrupts. • Handling website traffic. • Routers and switches in networking. • Maintaining the playlist in media players.	**2M** ***Any four applications 1/2M each***

	e) **Ans.**	**Write any four operations that can be performed on data structure** Data structure operations: • Insertion: Adding a new data. • Deletion: Removing a data. • Sorting: Arranging the data in some logical order. • Searching: Finding the location of data. • Traversal: Accessing each data exactly once. • Merging: Combining the data.	**2M** ***Any four operations 1/ 2M each***
	f) **Ans.**	**Convert the following infix expression to its postfix form using stack: A + B – C * D / E + F** Read Character | Stack Contents | Postfix expression A | | A + | + | A B | + | AB - | - | AB+ C | - | AB+C * | -* | AB+C D | -* | AB+CD / | -/ | AB+CD* E | -/ | AB+CD*E + | + | AB+CD*E/- F | + | AB+CD*E/-F | | **AB+CD*E/-F+**	**2M** ***Correct prefix expression 2M***
	g) **Ans.**	**Describe given two types of graphs: Directed graph and Undirected graph** **Directed Graph:** Directed graph is defined as the set of ordered pair of vertices and edges where each connected **edge has assigned a direction**. 	**2M** ***Each definition with diagram 1M***

		Undirected Graph: Undirected graph G is a graph in which each edge e is **not assigned a direction**. 	
2.	**a) Ans.**	**Attempt any <u>THREE</u> of the following:** **Describe working of selection sort method with suitable example** Working of selection sort: In selection sort, to sort data in ascending order 0^{th}element is compared with all other elements from the list. If the 0^{th}element is greater than compared element, then they are interchanged. In this way, the 0^{th}index element is compared with all elements till last element and if required then interchanged. At the end of pass 1, the smallest element is placed at 0^{th}index position. In the 2nd pass, 1^{st}index position element is compared with all elements placed in next index positions of a list. This process continues till all elements in a list are placed in sorted order. **Example:** Sort given input list in ascending order using selection sort input list: - 55, 25, 5, 15, 35. **Solution:** 	**12** **4M** ***Working 2M, Any Correct example 2M***

			
	b) **Ans.**	**Write algorithm to delete an intermediate node from a singly linked list.** **Algorithm:** Step 1 : Call to delete function with one argument as data. Step 2:Declare pointers temp,q. Step 3: Store address of first node in temp. Step 4: Repeat step 5 till search reaches to last node. Step 5: If info of next node of temp is data then 5.1 Store address of next node of temp in pointer q. 5.2 Set next field of temp node with address from next field of node q. 5.3 Delete node q using free function. Otherwise Increment pointer temp by one and Goto Step 4 Step 6: Return to calling function. OR	**4M** ***Correct algorithm with all steps 4M***

Step 1: start
Step 2: [is SLL empty?]
if start==NULL, then
Display "Empty List"
goto Step 7.
Step 3: [Accept node position (pos) to delete
node]
Step 4: [Traverse till pos^{th} and $(pos-1)^{th}$ node in SLL]
q=start
k=1
while(k<pos)
{
r=q;
q=q→next;
k++
}
Step 5: [Adjust pointers]
r→next=q→next
q→next=NULL
free(q)
Step 6: [Decrement node count by 1]
N=N-1
Step 7: stop

c) Ans. **Differentiate between tree and graph w.r.t any four parameters** **4M**

Any four points 1M each

Sr. No	Tree	Graph
1.	Tree is special form of graph i.e. minimally connected graph and having only one path between any two vertices.	In graph there can be more than one path i.e. graph can have unidirectional or bidirectional paths (edges) between nodes.
2.	Tree is a special case of graph having no loops, no circuits, and no self-loops.	Graph can have loops, circuits as well as can have self-loops.

		3.	Tree traversal is a kind of special case of traversal of graph. Tree is traversed in Pre-Order, In-Order and Post-Order.	Graph is traversed by DFS: Depth First Search and in BFS: Breadth First Search algorithm.
		4.	Different types of trees are: Binary Tree, Binary Search Tree, AVL tree.	There are mainly two types of Graphs: Directed and Undirected graphs.
		5.	Tree applications: sorting and searching like Tree Traversal & Binary Search.	Graph applications: Colouring of maps, in OR (PERT & CPM), algorithms, Graph colouring, job scheduling, etc.
		6.	Tree always has n-1 edges.	In Graph, no. of edges depends on the graph.
		7.	Tree is a hierarchical model.	Graph is a network model.

d) **Ans.**

Convert the given infix expression to postfix expression using stack and the details of stack at each step of conversion

Expression: A * B ↑ C – D / E +[F / G]

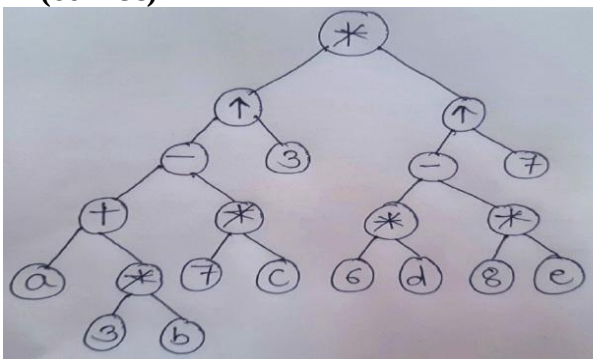
Note: Square brackets can be considered as round brackets

Infix ex	Read	Stack Contain	Postfix ex
A*B↑C-D/E+[F/G]	A	Empty	A
*B↑C-D/E+[F/G]	*	*	A
B↑C-D/E+[F/G]	B	*	AB
↑C-D/E+[F/G]	↑	*↑	AB
C-D/E+[F/G]	C	*↑	ABC
-D/E+[F/G]	-	-	ABC↑*
D/E+[F/G]	D	-	ABC↑*D
/E+[F/G]	/	-/	ABC↑*D
E+[F/G]	E	-/	ABC↑*DE
+[F/G]	+	+	ABC↑*DE/-
[F/G]	[	+[	ABC↑*DE/-
F/G]	F	+[	ABC↑*DE/-F
/G]	/	+[/	ABC↑*DE/-F
G]	G	+[/	ABC↑*DE/-FG
]	]	+	ABC↑*DE/-FG/
			ABC↑*DE/-FG/+

Postfix expression: ABC↑*DE/-FG/+

4M ***Correct answer with steps 4M***

3.		**Attempt any THREE of the following:**	**12**
	a)	**Explain complexity of following algorithms in terms of time and space:** **i) Binary Search** **ii) Bubble Sort**	**4M** ***Complexity of each 2M***
	Ans.	**Binary Search:** Space complexity is the total memory space required for execution of binary search algorithm. It depends on number of elements in a list and data type used to store the elements. Time complexity of binary search algorithm can be represented as f(n) where n is number of elements in an array. Complexity depends on number of comparison. In binary search, with each comparison the size of list of array elements is reduced to half. Consider number of elements in an array as n. After first comparison - $n/2=n/2^1$ After second comparison - $n/4=n/2^2$ After third comparison - $n/8=n/2^3$ After K^{th} comparison - $n/2^K$ In last comparison only one element will remain in list. So Maximum number of comparisons= number of element $n/2^K=1$ $n=2^K$ Place $\log_2$ on both sides: $\log_2 n=\log_2 2^K$ $\log_2 n=K$ $k=\log_2 n=O(\log_2 n)$ **Bubble sort:** Space complexity is the total memory space required for execution of bubble sort algorithm. It depends on number of elements in a list and data type used to store the elements. Time complexity depends on number of comparisons required for performing sorting. It requires N-1 passes to sort N elements in a list. In pass 1, N-1 comparisons are required to place largest element in its correct position. In pass2, N-2 comparisons are required to place second largest element in its correct position. To find time complexity, calculate total number of comparisons : $f(n) = (n-1)+(n-2)+(n-3)+...+3+2+1$ $=n(n-1)/2$	

		$= n^2/2+O(n) = O(n^2)$ So, time complexity of bubble sort is $O(n^2)$ where n is number of elements in an array.	
	b) **Ans**	**Draw an expression tree for the following expression:** **$(a+3b – 7c)^3 * (6d – 8c)^7$** 	**4M** ***Correct expression tree 4M***
	c) **Ans.**	**Describe working of bubble sort with example** In bubble sort, sorting begins with comparison of 0th index element with 1st index element in the list. If 0th index element is greater than 1st index element then numbers are interchanged. In this way, all elements are compared with its next element & interchanged if required. At the end of 1st pass, the largest element from the list is placed at the last position in an array. In pass 2, again comparison starts with 0th index element and 1st index element. If required then elements are interchanged. At the end of pass2, 2nd largest element is placed at its final position. This process of comparison and interchange continue in all passes. In every pass numbers of comparisons are reduced by one, as an element placed at its final position from bottom is not compared in next pass. This process continues till all elements are placed in sorted order. **Example: Input list 23, 15, 29, 11, 1** **Pass 1:** 23 → 15 → 15 → 15 → 15 15 → 23 → 23 → 23 → 23 29 → 29 → 29 → 11 → 11 11 → 11 → 11 → 29 → 1 1 → 1 → 1 → 1 → 29	**4M** ***Working 2M, Any Correct example 2M***

Pass 2:

15 15 15 15
23 23 11 11
11 11 23 1
1 1 1 23
29 29 29 29

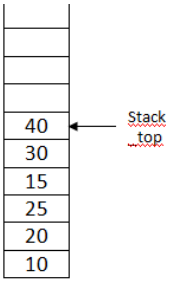
Pass 3:

15 11 11
11 15 1
1 1 15
23 23 23
29 29 29

Pass 4:

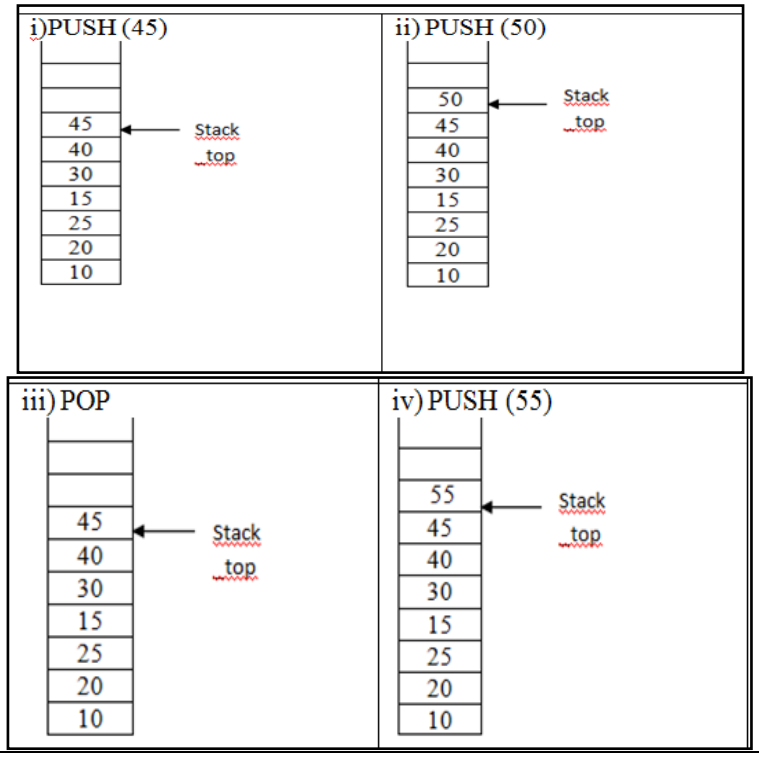
11 1
1 11
15 15
23 23
29 29

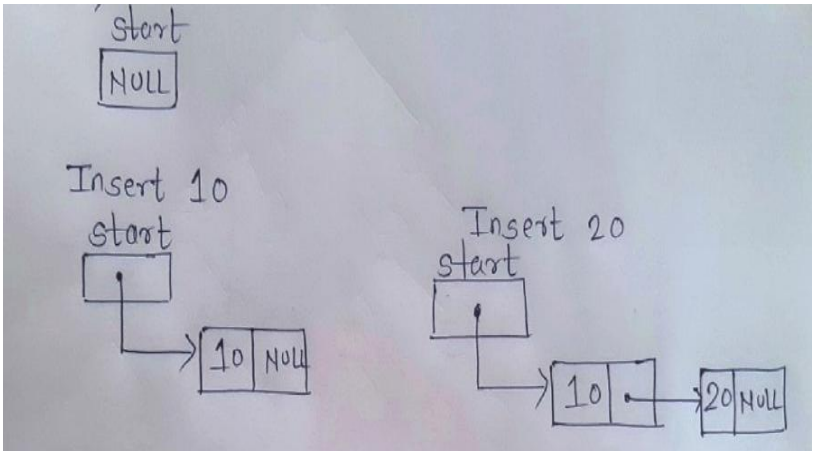
Sorted list : 1 ,11, 15, 23, 29

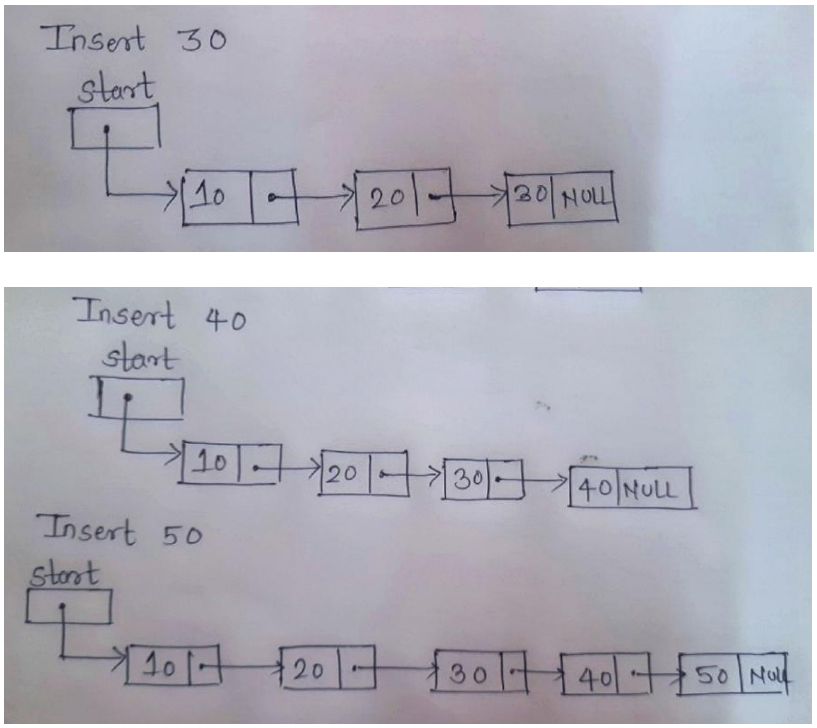
	d)	**Show the effect of PUSH and POP operation on the stack of size 10. The stack contains 10, 20, 25, 15, 30 and 40 with 40 being at top of stack. Show diagrammatically the effect of** **i) PUSH (45)** **ii) PUSH (50)** **iii) POP** **iv) PUSH (55)**	**4M**
	Ans.	Initial Array 	***Each correct effect 1M***

WINTER – 2023 EXAMINATION
MODEL ANSWER

Subject: Data Structure Using C **Subject Code:** **22317**



4.	**a)** **Ans.**	**Attempt any THREE of the following:** **Create singly linked list using data fields 10, 20, 30, 40, 50 and show step by step procedure with the help of diagram from start to end.** 	**12** **4M** ***Correct diagrams 4M***



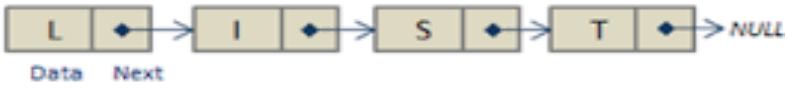
b) **Describe advantages of circular link list over linear link list with example** **4M**

Ans.

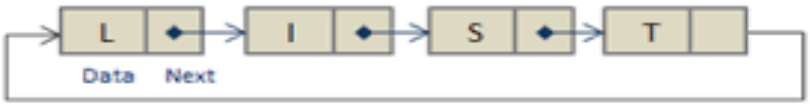
In circular linked list, last node contains address of first node so any node can be accesses again and again in a loop where as in linear linked list, traversing starts with head node and ends with NULL pointer in last node.

Description of any two advantages 2M each

Linear Linked list:



Circular linked list:



Advantages:

- Traversal is possible from node to any other node in the Circular linked list which was not possible in the singly linked list if we reached the last node.
- Traversal to head from the last node is easy.
- No need to use NULL pointer in a node in circular linked list. NULL value is used to indicate empty list.
- Circular linked list can be used with programs that requires traversal in loop such as round robin scheduling algorithm

c) Ans. **Explain the working of Radix sort method with an example** — **4M** ***Working 2M, Any Correct example 2M***

It is also called as bucket sort. This method use 10 buckets for sorting of numbers and 26 buckets for sorting character data(alphabets).
In pass 1, all elements are placed in buckets with respect to unit position digit. For example, if a number is 253 then this number is placed in bucket 3 as its unit position digit is 3. All elements from input list are placed in buckets. After placing all elements, all elements are retrieved from all buckets, starting from bucket 0 and used as a input list for pass 2. In pass 2, all elements are placed in buckets again but with respect to tenth position digit from each number. This process of placing elements in bucket and retrieving to form input list for next pass continues till all positions from input list are used to place elements in buckets.
Number of passes required by radix sort is equal to number of digits present in largest number from the input list. If the largest number is 253 then number of passes required are three as it includes unit position, tenth position and hundred position.

Example: Input list: 253,15,100,7,76
Pass 1: Unit position

	0	1	2	3	4	5	6	7	8	9
253				25**3**						
015						01**5**				
100	10**0**									
007								00**7**		
076							07**6**			

Retrieved list: 100,253,015,076,007

Pass 2: Tenth position

	0	1	2	3	4	5	6	7	8	9
100	1**0**0									
253						2**5**3				
015		0**1**5								
076								0**7**6		
007	0**0**7									

Retrieved list: 100,007,015,253,076

Pass 3: Hundred position

	0	1	2	3	4	5	6	7	8	9
100		**1**00								
007	**0**07									
015	**0**15									
253			**2**53							
076	**0**76									

Sorted List: 007,015,076,100,253

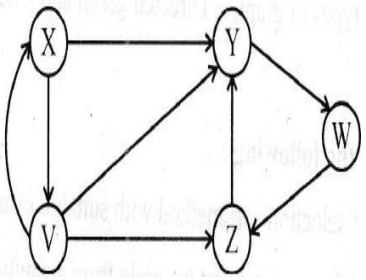
d) Ans. **Write an algorithm to count no. of nodes in singly linked list.** — **4M** ***Correct algorithm 4M***

Algorithm:

Step 1: Start / Call to count function
Step 2: Declare pointer temp
Step 3: Declare variable count and initialize it with value as 0
Step 4: Store address of first node from singly linked list in temp pointer
Step 5: Repeat step 6 till search/traversal reaches to last node
Step 6: Increment count by 1 and pointer temp by structure size (node size) and go to step 5
Step 7: Display value of variable count by adding 1 into it for last node.
Step 8: Stop / Return to calling function

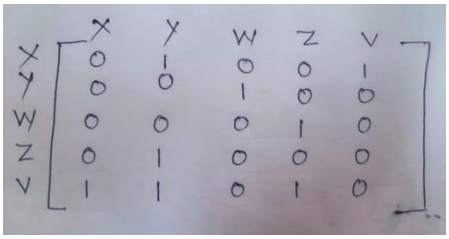
OR

Step 1: Start / Call to count function
Step 2: Declare pointer temp
Step 3: Declare variable count and initialize it with value as 0

		Step 4: Store address of first node from singly linked list in temp pointer Step 5: Repeat step 6 till temp is not NULL Step 6: Increment count by 1 and pointer temp by structure size (node size) and go to step 5 Step 7: Display value of variable count Step 8: Stop / Return to calling function OR Step 1: Start Step 2: [is SLL empty?] if start==NULL Display "Empty list" goto Step 5. Step 3: [Traverse till last node and count] q = start set count=0 while(q→next != NULL) count=count+1 q=q→next; Step 4: Display total count of nodes Step 5: Stop	
	e) **Ans.**	**Consider the graph ' G' in the following figure:**  **i) Find all simple path from X & Z** **ii) Find indegree and outdegree of node Y and Z** **iii) Find adjacency matrix A for the above graph** **iv) Give adjacency list representation of above graph** i) path 1 X-V-Z path 2 X-Y-W-Z path 3 X-V-Y-W-Z	**4M** ***Each sub question 1M***

ii) Indegree of Y - 3
outdegree of Y - 1
Indegree of Z - 2
outdegree of Z - 1

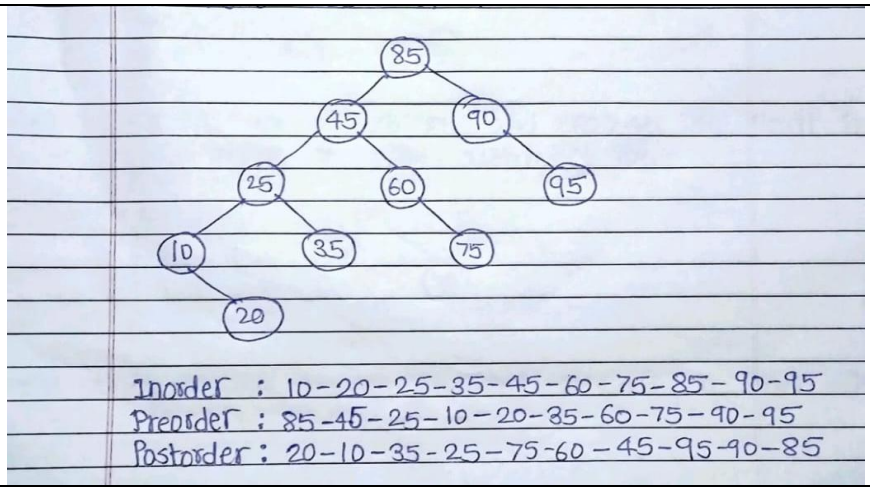
iii) Adjacency matrix :



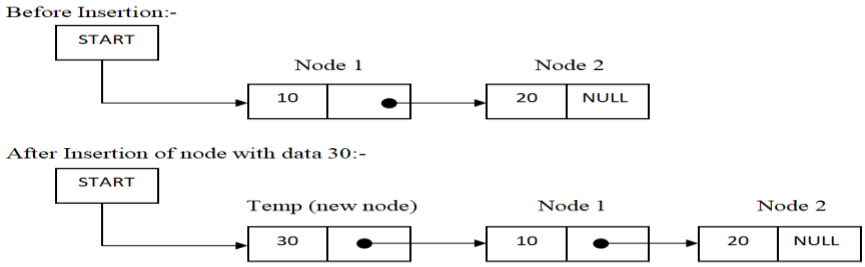
iv) Adjacency list:

Node	Adjacent Node
X	Y,V
Y	W
W	Z
Z	Y
V	X,Y,Z

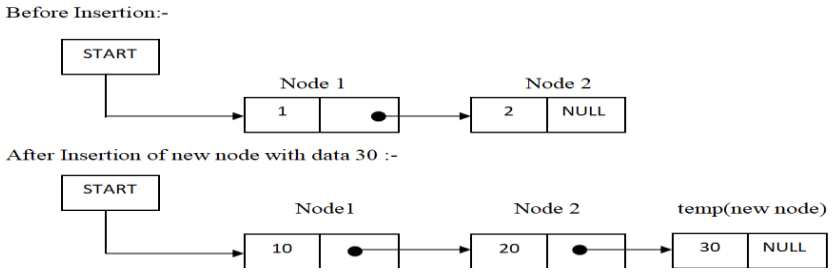
5.	**a)**	**Attempt any <u>TWO</u> of the following:** **Define the term tree traversal. Construct the Binary Search Tree (BST) of following:** **85, 90, 45, 60, 25, 35, 10, 20, 75, 95 and traverse the above BST in inorder, preorder & postorder.**	**12** **6M**
	Ans.	**Definition:** Tree traversal is defined as it is a way in which each node in the tree is visited exactly once in a particular order.	***Correct Definition 1M*** ***Correct Construction of BST 2M*** ***Correct traversal 1M for each***

			
	b) Ans.	**Explain operation on singly linked list.** ***Note: Description with diagram or algorithm or code shall be considered.*** There are various linked list operations that allow us to perform different actions on linked lists. Some of them are listed below: 1) Create a linked list 2) Insertion - adds a new element to the linked list 3) Traversal - access each element of the linked list 4) Deletion - removes the existing elements 5) Count – counting number of nodes in linked list 6) Search - Find a node in the linked list 7) Sort - sort the nodes of the linked list 1. **Creating Linked List: -**Creating a linked list insert first element (node) in linked list. Initially list contains start node which is a null pointer. Start NULL After inserting first element start pointer points to new node. 2. Inserting an element in linked list: - **2.1 Inserting a node at the beginning of list: -** In this operation, new node is created and added at the beginning of a list. To make new node first node, store existing node's address in new node's	**6M** ***Correct explanation of any three operations 6M***

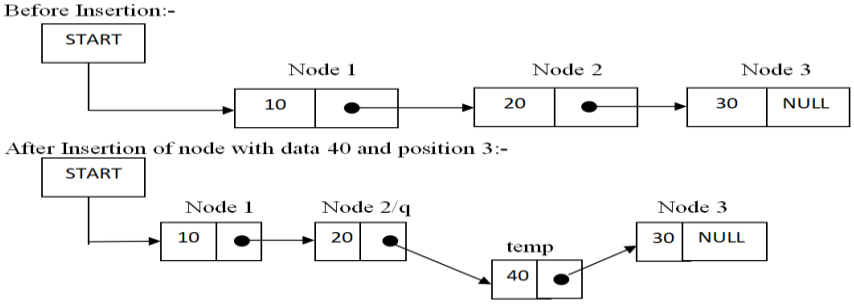
address field and add new node's address in header node.



2.2 Inserting a node at the end of the list: - In this operation, a new node can be added at the end of the list. Set new nodes info with data and next field with NULL value. Then search for the last node in a list and set last node's next field to new node.



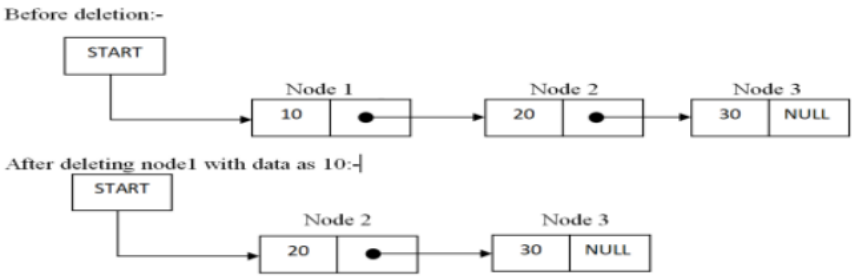
2.2 Inserting a node at the specified position: -In this operation, a new node can be added at any specified position. Set new node's info field with data, then traverse through the list up to the position -1 node (previous node) and set next field of previous as well as new node.



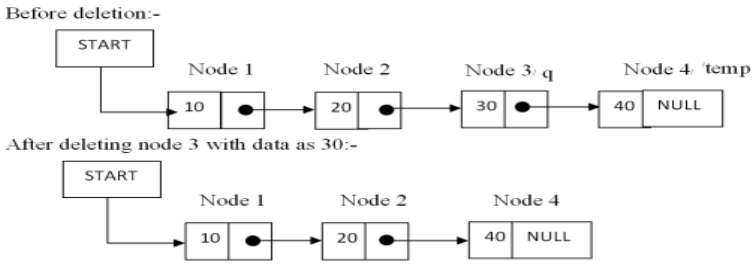
3. Display / traverse through linked list: - It is used to visit each and every element at-least once from the linked list.

4. Delete node from beginning/end/in between:

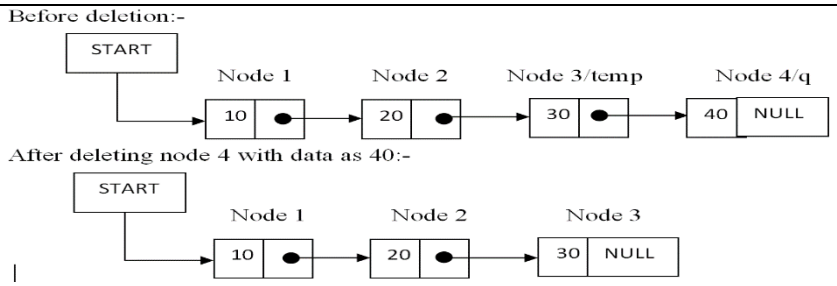
4.1 If data is the **first node** in a list then directly compare its INFO field with the data passed to the function and then set start pointer and delete first node.



4.2 If data is a node which is placed **in between** a list then traverse up to previous node then store the node to be deleted in a temporary node and set its previous node's pointer.



4.3 If the data is in the **last node,** then delete the data by traversing up to the last node and set its previous nodes NEXT field to NULL.



5. **Counting number of nodes from the list:** Counting number of nodes present in a linked list as linked list can store any number of nodes with dynamic memory allocation.

6. Search an Element on a Linked List: - Searching an element in a linked list is done using a loop. Store address of first node in temp node. Run a loop until the temp node is NULL because the last element points to NULL. In each iteration, check if the search element is equal to info of node. If it is equal then return true otherwise return false.

7. Sort Elements of a Linked List: - Store address of first node in temp node and create another node index for later use. Run a loop till the last node (i.e. NULL).In each iteration, follow the following step. Store the next node of temp in index. Check if the data of the temp node is greater than the next node. If it is greater, swap temp and index.

c) **Implement a 'C' program to insert element into the queue and delete the element from the queue.** **6M**

Ans. **Note: Any other logic shall be considered.**

Correct logic for insertion 3M

Correct logic for deletion 3M

```
#include <stdio.h>
#include<conio.h>
#define max 5
void main()
{
int a[max],front,rear,i,no,choice;
front=-1;
rear=-1;
do
{
```

```
printf("Menu: 1.Insert 2.Delete 3.Display");
printf("\n Enter your choice:");
scanf("%d",&choice);
switch(choice)
{
case 1:
printf("\n Enter number");
scanf("%d",&no);
if(rear==max-1)
{
printf("\n Queue is full");
break;
}
rear=rear+1;
a[rear]=no;
if(front==-1)
front=0;
break;

case 2:
if(front==-1)
{
 printf("\n queue is empty");
 break;
}
no=a[front];
if(front==rear)
front=rear=-1;
else
front=front+1;
printf("\n Element deleted from queue is %d",no);
break;
case 3:
printf("\n Queue elements are:");
for(i=front;i<=rear;i++)
printf("\n%d",a[i]);
break;
default:
printf("\n Invalid Choice");
```

```
}
printf("\n Do you want to continue:(1 for yes/2 for no)");
scanf("%d",&choice);
}while(choice==1);
getch();
}
```

6. **Attempt any TWO of the following:** **12**

a) Find out prefix equivalent of the expression , **6M**

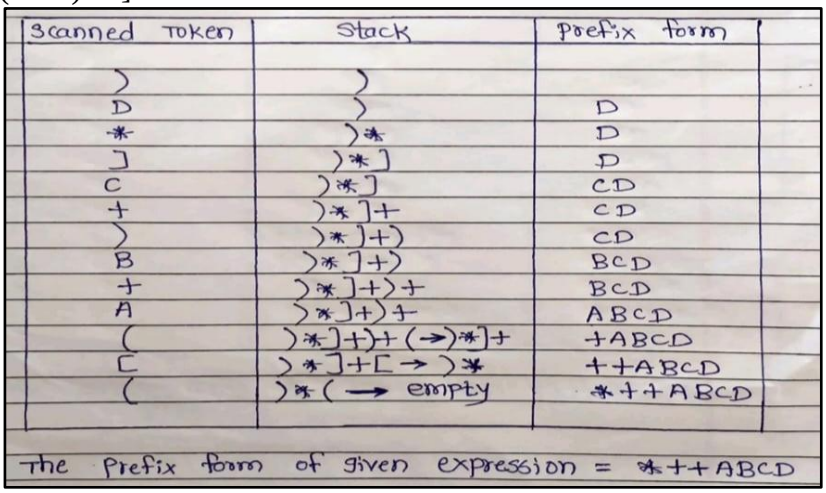
i. [(A+B)+C]*D

ii. A[(B * C) + D]

Note: Square brackets can be considered as round brackets

Ans.

i. [(A+B)+C]*D



Scanned token	Stack	Prefix form
)	)	
D	)	D
*	)*	D
]	)*]	D
C	)*]	CD
+	)*]+	CD
)	)*]+)	CD
B	)*]+)	BCD
+	)*]+)+	BCD
A	)*]+)+	ABCD
(	)*]+)+ (→)*]+	+ABCD
[	)*]+[→)*	++ABCD
(	)*(→ empty	*++ABCD

The prefix form of given expression = *++ABCD

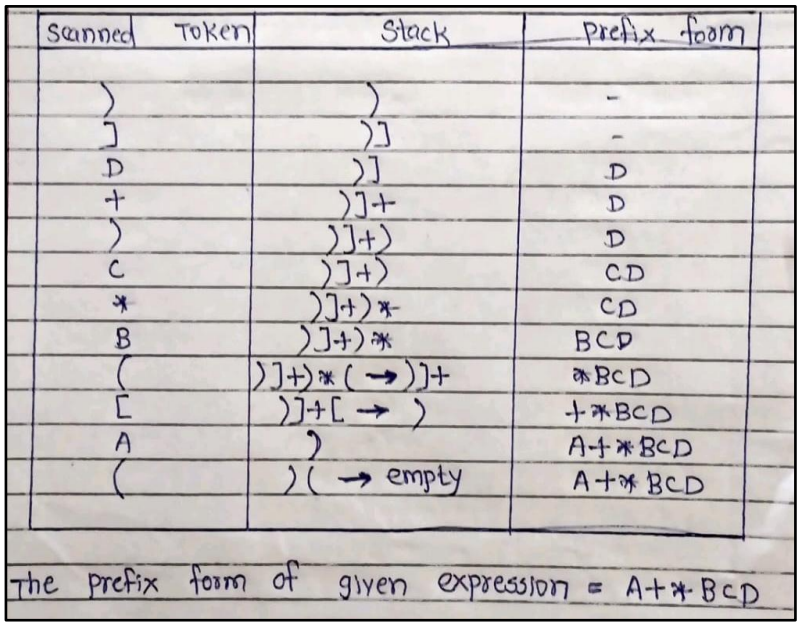
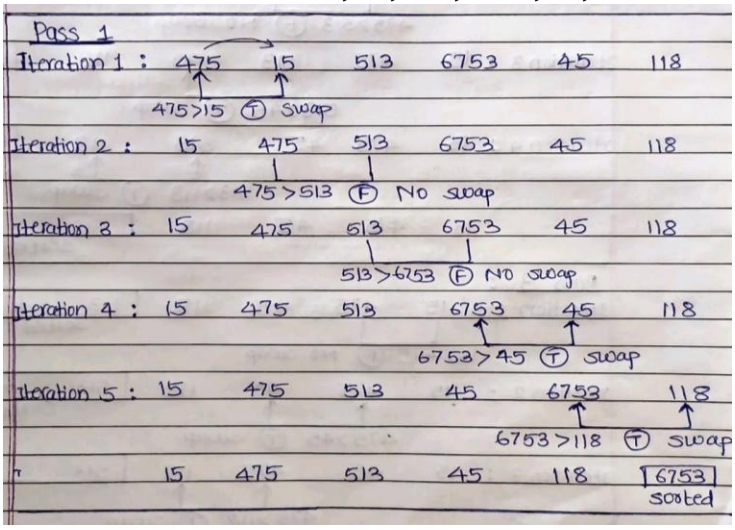
Correct prefix expression 3M each

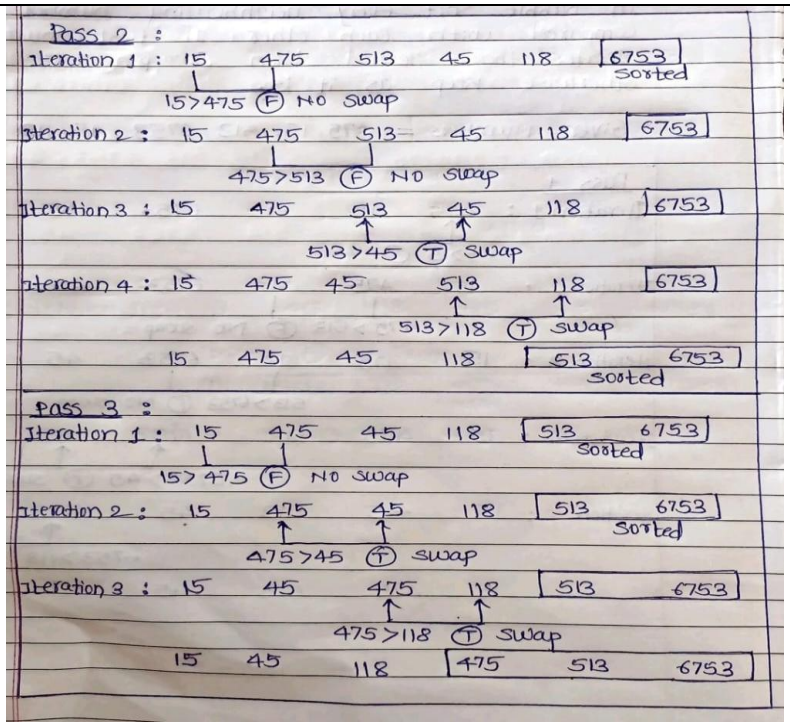
ii)A[(B * C) + D]

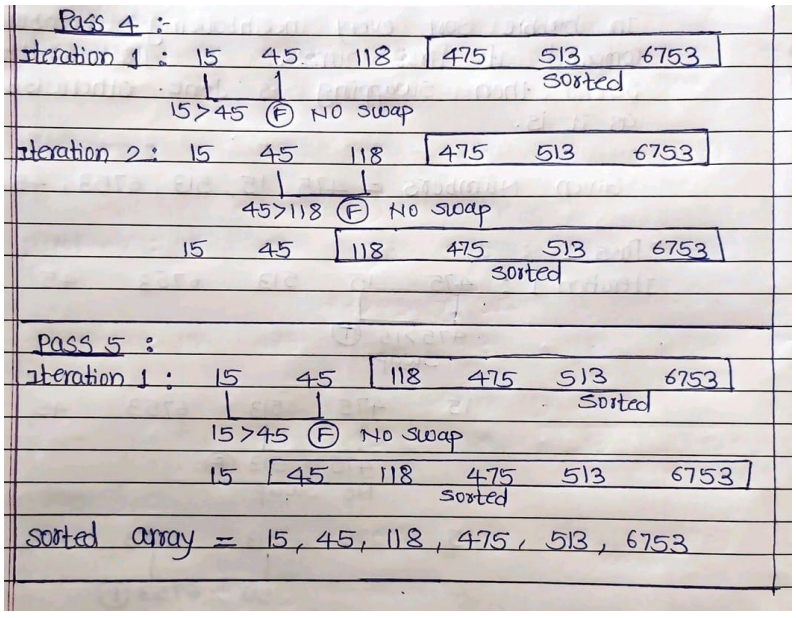
Note: If question is considered as A*[(B*C)+D] then marks shall be given to the answer as *A+*BCD

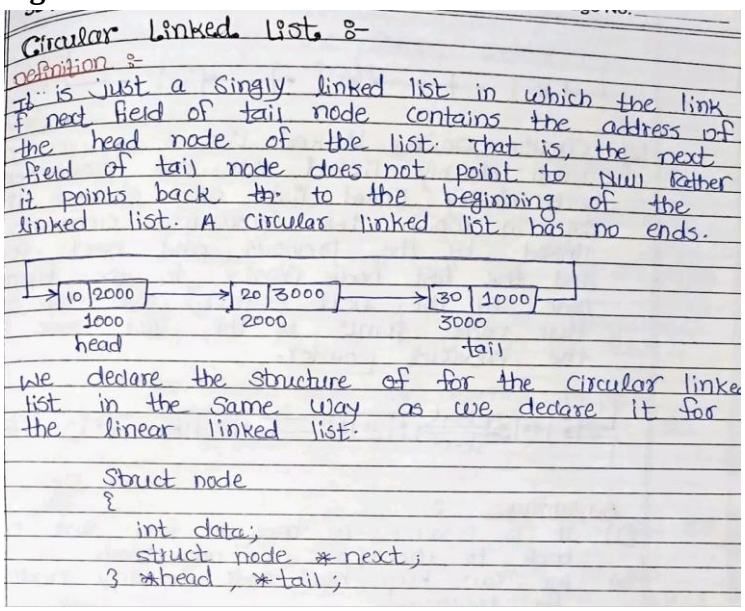
WINTER – 2023 EXAMINATION
MODEL ANSWER

Subject: Data Structure Using C **Subject Code:** **22317**

			
	b) **Ans.**	**Sort the following number in ascending order using bubble sort. Given numbers as follows : 475, 15, 513, 6753, 45, 118.** 	**6M** ***Correct steps of sorting 6M***


Pass 2 :
Iteration 1 : 15 475 513 45 118 6753
sorted
15>475 (F) NO swap
Iteration 2 : 15 475 513 45 118 6753
475>513 (F) NO swap
Iteration 3 : 15 475 513 45 118 6753
513>45 (T) swap
Iteration 4 : 15 475 45 513 118 6753
513>118 (T) swap
15 475 45 118 513 6753
sorted
Pass 3 :
Iteration 1 : 15 475 45 118 513 6753
sorted
15>475 (F) NO swap
Iteration 2 : 15 475 45 118 513 6753
sorted
475>45 (T) swap
Iteration 3 : 15 45 475 118 513 6753
475>118 (T) swap
15 45 118 475 513 6753


Pass 4 :-
Iteration 1 : 15 45 118 475 513 6753
sorted
15>45 (F) NO swap
Iteration 2 : 15 45 118 475 513 6753
45>118 (F) NO swap
15 45 118 475 513 6753
sorted
Pass 5 :
Iteration 1 : 15 45 118 475 513 6753
sorted
15>45 (F) NO swap
15 45 118 475 513 6753
sorted
sorted array = 15, 45, 118, 475, 513, 6753

	c) **Ans.**	**Describe circular linked list with suitable diagram. State advantages of circular linked list over linear linked list.**  Circular Linked List :- definition :- It is just a singly linked list in which the link f next field of tail node contains the address of the head node of the list. That is, the next field of tail node does not point to Null rather it points back to the beginning of the linked list. A circular linked list has no ends. 10 2000 → 20 3000 → 30 1000 1000 2000 3000 head tail We declare the structure of for the circular linked list in the same way as we declare it for the linear linked list. Struct node { int data; struct node *next; } *head, *tail; **Advantages of Circular Linked Lists:** 1) Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again. 2) Useful for implementation of queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last. 3) Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list. 4) Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap	**6M** ***Explanation of circular linked list 2M*** ***Diagram 2M*** ***Any two Advantages 2M***